Node.js Internals

Complete Interview Revision Guide

Prepared for **Satish Kumar**

SDE-3 Node.js — WeKan Enterprise Solutions

August 2026

Every concept is explained through one running example — a tea counter run by a single waiter named Ramesh — and every section ends with the exact sentence to say in an interview.

Contents

1. **The Foundation — V8, libuv and the Event Loop**
   a. The one-waiter shop
   b. What V8 is and what it is not
   c. What libuv is
   d. The two async paths — kernel vs thread pool
   e. The full architecture diagram
   f. The event loop's six phases
   g. The two queue-jumpers — nextTick and Promises
   h. setTimeout vs setImmediate
2. **Blocking and CPU-bound Work**
3. **Streams and Backpressure**
4. **Error Handling**
5. **Memory and Garbage Collection**
6. **EventEmitter**
7. **Cluster**
8. **Buffers**
9. **Modules — CommonJS vs ECMAScript Modules**
0. **Timers**
1. **MongoDB — Indexes and Query Performance**
2. Appendix A — Acronyms expanded
3. Appendix B — Every one-liner in one place
4. Appendix C — Rapid-fire self test

How to read this document

- **Green boxes** are the shop analogy — read these first if a concept is not landing.
- **Yellow boxes** are the specific questions you asked during the session, answered in place.
- **Blue bars** are the exact sentences to say out loud in the interview.
- **Red bars** are traps and common mistakes.

1. The Foundation — V8, libuv and the Event Loop

This is the chapter to read twice. Everything else in Node.js is a consequence of what happens here.

1a. The one-waiter shop

> **THE RUNNING EXAMPLE**
> House of Rinish has a small tea counter. There is **exactly one waiter, Ramesh**. Not two. Not four. This never changes.
>
> Three customers arrive at the same moment:
> • **Priya** wants chai. It must boil. 3 minutes.
> • **Rahul** wants a packed biscuit off the shelf. 5 seconds.
> • **Meena** wants coffee. It must brew. 3 minutes.

The bad waiter

Ramesh takes Priya's order, puts the chai on the stove, and **stands there watching the pot for 3 minutes**. Rahul waits. Meena waits.

Rahul wanted a 5-second biscuit. He got it after 3 minutes. The shop is slow, and it is slow for a stupid reason: the waiter was standing still.

The Node.js waiter

Ramesh takes Priya's order, puts the chai on the stove, and **walks away immediately**. The stove will beep when it is done.

He turns to Rahul — biscuit, 5 seconds, done. He turns to Meena — coffee on the second stove, walks away. Now he stands free.

Beep — Priya's chai. He serves it. *Beep* — Meena's coffee. He serves it.

> **THE WHOLE IDEA**
> Same one waiter. Everyone served fast. **Ramesh never waited for anything.** Node.js is not fast because of magic. It is fast because it never wastes a thread standing still.

Naming the parts

In the shop	In Node.js
Ramesh, the single waiter	The **main thread** — all your JavaScript runs here
Ramesh's hands and brain	**V8** — the engine that executes JavaScript
The stove that beeps by itself	The **OS kernel** — watches network connections for free
Four helpers in the back room	The **libuv thread pool** — 4 threads for file and crypto work
Ramesh asking "anyone ready?"	The **event loop**
"Serve Priya her chai" — the pending task	A **callback**

1b. What V8 is — and what it is not

V8 is Google's JavaScript engine. Same one inside Chrome. Its entire job is: take JavaScript text, turn it into machine code, run it.

V8 owns two things:

- **The call stack** — a pile of plates. A function starts, a plate goes on. The function returns, the plate comes off. There is **one** stack. This is what "Node is single-threaded" actually means.
- **The heap** — where your objects, arrays, strings and closures live. Garbage-collected.

> **The part people miss:** V8 knows **nothing** about files, networks, timers or servers. Open the V8 source and you will not find `fs`, `http` or `setTimeout`. Those are not JavaScript. They are Node's additions.
>
> Run JavaScript in a browser and you get `document` and `fetch`. Run it in Node and you get `fs` and `http`. **Same V8, different surroundings.** That difference is the point of the next section.

1c. What libuv is

libuv is a C library. The name means *library for Unified asynchronous I/O*. It was written for Node and is where Node's actual power lives.

If V8 is Ramesh's brain, libuv is the entire back of house — the stoves, the helpers, and the schedule that tells Ramesh where to go next.

libuv provides three things:

1. **The event loop itself** — the loop Ramesh walks
2. **Non-blocking network I/O** — by talking to the operating system kernel
3. **A thread pool** — 4 real threads, for work the kernel cannot do asynchronously

Between your JavaScript and libuv's C code sits a thin layer of **Node C++ bindings**. When you write `fs.readFile(...)`, a binding translates

that into the matching libuv C function.

> **SAY THIS**
> "Node is not single-threaded — the *JavaScript execution* is single-threaded. libuv underneath is multi-threaded, and network I/O does not even use those threads because the kernel handles it."
>
> This one sentence separates you from candidates who recite "Node is single-threaded and non-blocking."

1d. The two async paths

Here is the detail interviewers actually probe. There are **two completely different routes** for background work, and they cost completely different amounts.

Path A — the kernel path. Zero threads.

Modern operating systems have a built-in ability to watch thousands of network connections at once and report which ones have data. On Linux this is called **epoll**. On macOS, **kqueue**. On Windows, **IOCP** (I/O Completion Ports).

So libuv does not need to do anything clever. It registers the socket with the kernel and says "*wake me when data arrives*", then forgets about it.

This costs zero threads. This is why one Node process can hold ten thousand open connections. Each one is just another pot on a stove that watches itself.

Path A covers: HTTP requests in and out, TCP sockets, MongoDB queries, Redis calls, pipes, and `dns.resolve`.

Path B — the thread pool. Four threads.

Some work has no self-watching kernel equivalent. Reading a file from disk is the main one — there is no reliable OS-level "tap me when this file is read" for ordinary files. Somebody has to physically sit and read it.

So libuv keeps **four background threads** (configurable via the `UV_THREADPOOL_SIZE` environment variable) and hands the job to one of them.

Path B covers: everything in `fs`, `crypto` functions like `bcrypt` and `pbkdf2`, `zlib` compression, and `dns.lookup`.

> **NOTE THIS PAIR**
> `dns.lookup` uses a **thread pool thread**. `dns.resolve` uses the **kernel path**. They sound identical and behave completely differently. This exact pair is a favourite interview question.

The practical consequence

Priya's, Rahul's and Meena's browser connections cost **zero threads**. But if your login handler calls `bcrypt.hash()`, each call takes one of only four helpers.

Eight simultaneous logins, each hashing for 100ms: four are hashed immediately. The fifth through eighth sit in a queue and wait an extra 100ms. Not because Node is slow — because there are only four helpers.

What you write	Path	Threads used
MongoDB query	Kernel	**0**
HTTP call to a payment gateway	Kernel	**0**
Incoming browser connection	Kernel	**0**
`dns.resolve`	Kernel	**0**
`fs.readFile("invoice.pdf")`	Thread pool	**1 of 4**
`bcrypt.hash(password)`	Thread pool	**1 of 4**
`zlib.gzip(data)`	Thread pool	**1 of 4**
`dns.lookup`	Thread pool	**1 of 4**

> **Trap question:** "Should we raise `UV_THREADPOOL_SIZE` to 64 so our API handles more concurrent HTTP requests?"
>
> **Wrong premise.** HTTP requests never touch the thread pool. Raising it would help file and crypto throughput and would do nothing at all for HTTP concurrency.

1e. The full picture

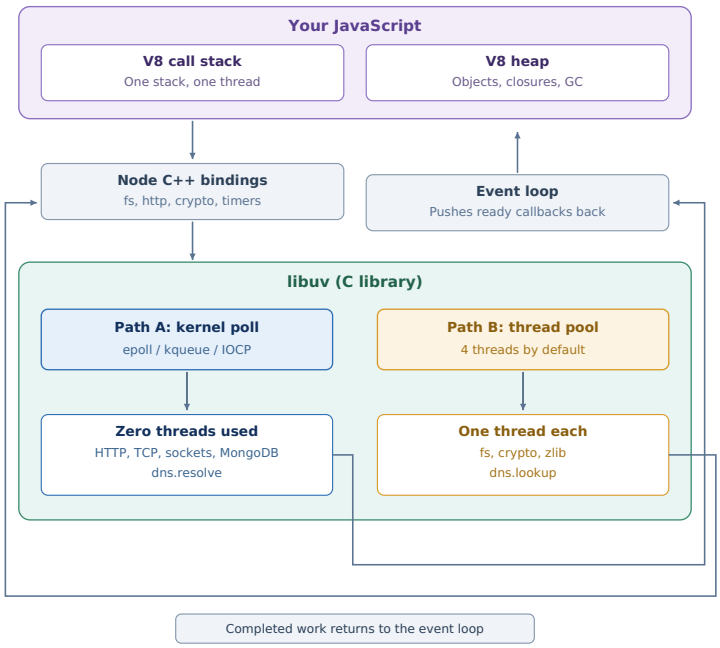

Figure 1 — The complete Node.js runtime. Note both return arrows: work from either path comes back through the event loop.

Priya's order, step by step

1. Priya's request arrives. The kernel wakes libuv. The event loop pushes your Express handler onto V8's call stack.
2. The handler runs `await db.orders.insertOne({amount: 2499})`. That is network I/O — Path A. libuv registers the socket and **the handler's stack frame is popped**. The thread is free.
3. Rahul's and Meena's handlers now run on that same free thread. Both also hit MongoDB.
4. MongoDB replies for Rahul first. The event loop resumes Rahul's continuation.
5. Then Priya's. Then Meena's.

Figure 2 — The single thread works in short bursts. Total elapsed time is 3 seconds, not 6.

1f. The event loop's six phases

Most people imagine one queue: work finishes, goes in a queue, gets executed. **That is wrong.**

Ramesh walks a **fixed circuit** past six counters, in the same order, forever. **Each counter has its own separate queue.** He clears everything at one counter before moving to the next.

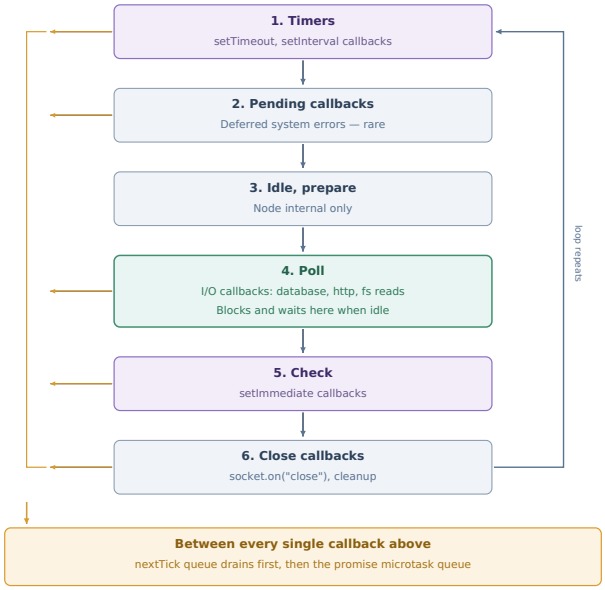

Figure 3 — The six phases. Orange arrows show the queue-jumpers running between every callback in every phase.

In practice only three phases matter: timers, **poll** and **check**. Name all six to show you know the list, then talk about these three.

Where each thing lands

You write	Handled at
`setTimeout(fn, 100)`	Timers — phase 1
MongoDB result, `fs.readFile` result, incoming request	Poll — phase 4
`setImmediate(fn)`	Check — phase 5
`socket.on("close")`	Close — phase 6
`process.nextTick(fn)`	**Not a phase** — jumps the queue
`.then(fn)` / code after an `await`	**Not a phase** — jumps the queue

1g. The two queue-jumpers

`process.nextTick` and promise callbacks do **not** wait their turn in the circuit. They are the manager standing behind Ramesh saying: "*before you take one more step, finish this.*"

After **every single callback** Ramesh executes, anywhere in the circuit, he must:

1. Drain the entire `nextTick` queue
2. Then drain the entire promise microtask queue
3. Only then continue his round

nextTick beats promises. Both beat everything in the circuit.

```
console.log("A");

setTimeout(() => console.log("B"), 0);            // timers phase
setImmediate(() => console.log("C"));             // check phase
Promise.resolve().then(() => console.log("D"));   // microtask — jumper
process.nextTick(() => console.log("E"));         // nextTick — top priority

console.log("F");
```

Output: **A, F, E, D, B, C**

Walk it: `A` and `F` are plain synchronous code, executed immediately with no queue involved. The current chunk of work then ends, so the jumpers fire — `E` first because nextTick beats promises, then `D`. Only now does Ramesh start his circuit: timers gives `B`, check gives `C`.

THE MISTAKE YOU MADE — WORTH REMEMBERING

Given a `process.nextTick` written **inside** a `setTimeout` callback, you placed it before the setTimeout's own log. It cannot be.

Queue-jumping is about priority, not time travel. A nextTick registered at time T runs at the next checkpoint after T. It cannot run before the line of code that created it. So the setTimeout callback logs first, *then* registers the nextTick, *then* the nextTick fires — still beating everything scheduled later, like setImmediate.

Starving the event loop. Because nextTick always jumps ahead, a nextTick that schedules another nextTick never lets Ramesh move:

```
function boom() { process.nextTick(boom); }
boom();   // server dead, 100% CPU, no requests served
```

This is why the practical advice is: **use `setImmediate` unless you have a specific reason for `nextTick`.**

1h. setTimeout(fn, 0) vs setImmediate

At the **top level of a file**, the order between these two is **non-deterministic**. It depends on how fast the process started. Sometimes one, sometimes the other.

But **inside an I/O callback, `setImmediate` always wins**:

```
fs.readFile("a.txt", () => {          // we are now in the poll phase
  setTimeout(() => console.log("timeout"), 0);
  setImmediate(() => console.log("immediate"));
});
// always: immediate, then timeout
```

Why: Ramesh is standing at poll (phase 4). Check (phase 5) is his very next stop. Timers (phase 1) requires a full lap. So `setImmediate` is guaranteed to run first here.

SAY THIS

"The event loop is a loop that continuously checks the queues of completed async work and pushes their callbacks onto the call stack, one at a time, in a fixed order of phases — timers, pending, idle/prepare, poll, check, close. Between every callback it drains the nextTick queue and then the promise microtask queue, which is why those two always run before anything else scheduled."

2. Blocking and CPU-bound Work

> **THE SHOP**
> A customer asks Ramesh to count a jar of ten lakh coins. There is **no stove for counting coins**. He must do it himself, standing at the counter.
>
> For those ten minutes, **nobody else gets served**. Not Priya, not Rahul, not the 500 people who walk in. The shop is frozen.

```
app.get("/report", (req, res) => {
  let total = 0;
  for (let i = 0; i < 1e9; i++) total += i;   // Ramesh counting coins
  res.json({ total });
});
```

While this runs, **every other request to your entire server is stuck**. This is the number one Node.js production bug.

> **The trade-off you accept by choosing Node.** In Spring Boot, Tomcat gives each request its own thread — one slow request hurts one user. In Node, one slow function hurts **everyone on that process**.

How to recognise it

Ask one question: **is the code *waiting*, or is it *doing*?**

- Waiting on MongoDB or an API — **fine**, that is a stove
- Adding up ten lakh numbers — **problem**
- `JSON.parse()` on a 40 MB payload — **problem**, and easy to miss
- Image resizing, PDF generation, CSV parsing — **problem**
- Anything whose name ends in `Sync` — **problem**
- A badly written regex that backtracks — **problem** (called ReDoS)

How to detect it in production

The metric is **event loop lag**. Set a timer for 500ms and measure how late it actually fires. The gap *is* the lag.

```
let last = Date.now();
setInterval(() => {
  const lag = Date.now() - last - 500;
  if (lag > 100) console.warn(`Event loop lag: ${lag}ms`);
  last = Date.now();
}, 500);
```

In a real system use `perf_hooks.monitorEventLoopDelay()` and alarm on the 99th percentile.

The four fixes, in the order to consider them

Fix 1 — Do not do it in Node at all

The bad way. You want total sales per city:

```
const orders = await db.orders.find({}).toArray();   // 5 lakh docs into memory

const totals = {};
for (const o of orders) {                            // Ramesh counting coins
  totals[o.city] = (totals[o.city] || 0) + o.amount;
}
```

Two separate problems: 5 lakh documents now sit in Node's heap, **and** Ramesh personally loops through all of them.

The good way. MongoDB already knows how to group and sum:

```
const totals = await db.orders.aggregate([
  { $group: { _id: "$city", total: { $sum: "$amount" } } }
]).toArray();

// [ { _id: "Jamshedpur", total: 284500 },
//   { _id: "Ranchi",     total: 191200 } ]
```

MongoDB does the counting on its own machine. For Ramesh this is now a **stove job**, not a coins job.

> **THE PRINCIPLE**
> Move work to whoever is built for it. Databases are built for grouping and sorting. Node is built for handling connections. **Do not make Node do the database's job.** Say this first in an interview — most candidates jump straight to worker threads.

Fix 2 — Chunk it, so Ramesh takes breaks

Sometimes Node genuinely must do the work — generating 5 lakh invoice PDFs, for instance.

```
function makeInvoices(orders, i = 0) {
  const stop = Math.min(i + 1000, orders.length);

  for (; i < stop; i++) {
    makeInvoice(orders[i]);                          // do 1,000
  }

  if (i < orders.length) {
    setImmediate(() => makeInvoices(orders, i));     // break, then resume
  }
}
```

`setImmediate` tells Ramesh: "*stop, do a lap of the shop, serve whoever is waiting, then come back to the jar.*"

Instead of one ten-minute freeze you get 500 tiny bursts with customers served in between. The total job takes slightly longer; **nobody is locked out**.

> Must be `setImmediate`, **not** `nextTick`. nextTick is a queue-jumper — it would send Ramesh straight back to the jar without letting him serve anyone. Same freeze, no break.

Fix 3 — worker_threads

Hire a second Ramesh in a back room.

```
// main.js
const { Worker } = require("worker_threads");

app.get("/report", (req, res) => {
  const worker = new Worker("./count.js");
  worker.on("message", (result) => res.json(result));
  worker.on("error",   (err)    => res.status(500).json({ error: err.message }));
});

// count.js — the back room
const { parentPort } = require("worker_threads");
let total = 0;
for (let i = 0; i < 1e9; i++) total += i;
parentPort.postMessage({ total });
```

Front Ramesh hands over the jar and **goes straight back to serving**. When the back room finishes, it shouts the total.

> **A worker is not just a background thread.** It is a completely separate V8 instance with its own heap and its own event loop, inside the same process. Variables are **not** shared — data passed via `postMessage` is **copied**. That is why you cannot simply read a variable from the main file.

Two things that show maturity:

- **Never create a worker per request.** Spawning costs 10–30ms plus its own memory. Under load you would create thousands. Use a **worker pool** — the `piscina` library, or N workers with a job queue. Same idea as a database connection pool.
- **Measure first.** If the work takes 5ms, copying the data to the worker costs more than the work itself. Workers are for hundreds of milliseconds and up.

Fix 4 — Move it out of the request path

Push a job onto RabbitMQ, let a separate worker service handle it, notify the user when done. For anything beyond a few hundred milliseconds this is the real production answer.

worker_threads vs cluster vs child_process

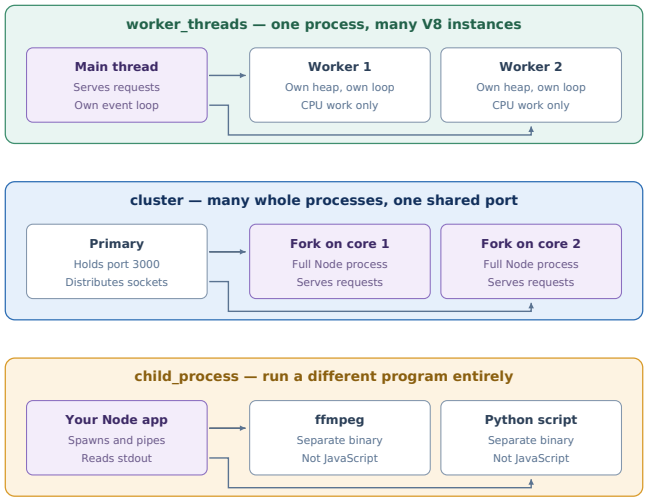

Figure 4 — Three different tools for three different problems.

Tool	Solves
`worker_threads`	**CPU-bound work.** Threads in one process, cheap to communicate with, memory genuinely shareable via `SharedArrayBuffer`.
`cluster`	**Not using all your CPU cores.** An 8-core box runs one Node process on one core; cluster forks 8 so all cores serve traffic.
`child_process`	**Needing a different program.** ffmpeg, a Python ML script, a shell command.

Trap: "We have slow endpoints, should we use cluster?"
Cluster multiplies **capacity**, it does not fix a **blocked loop**. If one request blocks for 2 seconds, cluster with 8 workers means 8 requests block for 2 seconds each instead of 1. It helps throughput, not latency. **Fix the blocking first.**

THE ASSEMBLED ANSWER

"First I would confirm it is actually CPU-bound by checking event loop lag — I do not want to solve the wrong problem. Then, in order: can the database do this work instead, via an aggregation pipeline? If not, and the user is waiting on it, I would move it to a worker thread from a pooled set of workers, not a new worker per request. If it is genuinely long-running, it does not belong in the request path at all — I would push it onto a queue and notify asynchronously, which is the pattern I have used with RabbitMQ."

3. Streams and Backpressure

The problem

A customer downloads their 2 GB order history CSV.

```
app.get("/export", (req, res) => {
  const data = fs.readFileSync("orders.csv");   // 2 GB into memory
  res.send(data);
});
```

Ramesh picks up the entire 2 GB, holds it, then hands it over. Two problems: **2 GB sits in your process**, and `readFileSync` is Ramesh doing it himself, so the shop is frozen throughout.

The fix

```
app.get("/export", (req, res) => {
  fs.createReadStream("orders.csv").pipe(res);
});
```

Two lines. Memory usage: about **64 KB**, not 2 GB.

> **THE SHOP**
> Ramesh does not carry the whole sack of rice. He carries it **one small bowl at a time** — fill a bowl, hand it over, fill the next. And between bowls he is **free**, so he goes around the loop and serves other customers.

The file is read in **chunks** (64 KB by default). Each chunk goes straight out and is then discarded. Only one chunk is in memory at any moment. Memory stays constant **regardless of file size**.

Backpressure — the part interviews test

Reading from disk is **fast**. A customer's mobile connection is **slow**.

If Ramesh keeps filling bowls faster than the customer takes them, bowls pile up on the counter. That pile is memory. Enough pile-up and you are holding 2 GB again — you have solved nothing.

Backpressure is the consumer saying "stop, I am not ready."

```
const ok = writable.write(chunk);
if (!ok) {
  readable.pause();
  writable.once("drain", () => readable.resume());
}
```

When the writable's internal buffer fills, `write()` returns `false`. That means pause. When it drains, a `drain` event fires. That means resume.

> **You almost never write this yourself.** `pipe()` does all of it for you. That is the entire reason `pipe()` exists.

pipeline() beats pipe()

`pipe()` has a real flaw: **if one stream errors, the others are not cleaned up**. File handles stay open, memory stays held.

```
// leaks on error
fs.createReadStream("in.csv").pipe(gzip).pipe(fs.createWriteStream("out.gz"));
```

```
// cleans up everything, one error callback
const { pipeline } = require("stream");

pipeline(
  fs.createReadStream("in.csv"),
  zlib.createGzip(),
  fs.createWriteStream("out.gz"),
  (err) => { if (err) console.error("failed:", err); }
);
```

> **YOUR QUESTION — WHERE PIPE AND PIPELINE FIT WITH EVENTEMITTER ERRORS**
> That chain has **three streams — three separate EventEmitters**. As covered in chapter 4, an EventEmitter that emits `error` with no listener **crashes the process**. So with `.pipe()` you must attach an error listener to *every one*, and miss a single one and you crash.
>
> `pipeline()` does three things for you: attaches an error listener to every stream, destroys all of them if any one fails, and reports through a single callback.
>
> **Always use `pipeline()` in production.** Saying this unprompted marks you as someone who has debugged real Node systems.

The four stream types

Type	Does	Example
Readable	Source of data	`fs.createReadStream`, HTTP request
Writable	Destination	`fs.createWriteStream`, HTTP response
Duplex	Both, independently	TCP socket
Transform	Both, modifies in between	`zlib.createGzip`, a CSV parser

Transform is the useful one — it is how you chain work:

```
pipeline(
  fs.createReadStream("orders.csv"),
  csvParser(),           // Transform: text to objects
  filterJamshedpur(),    // Transform: drop other cities
  zlib.createGzip(),     // Transform: compress
  res,
  handleError
);
```

2 GB in, filtered and compressed out, memory stays at a few hundred KB. Never more than a few chunks in flight.

Where this connects to MongoDB

Direct relevance to WeKan, who do enterprise data migrations. **A MongoDB cursor is a readable stream.**

```
// bad — 5 lakh documents into memory
const orders = await db.orders.find({}).toArray();

// good — one document at a time
const cursor = db.orders.find({}).stream();
pipeline(cursor, csvFormatter(), res, handleError);
```

> **SAY THIS — MIGRATING 20 LAKH ROWS**
>
> "With `insertMany` I would hold all 20 lakh documents in memory first, risking a heap out-of-memory crash, and nothing gets written until everything is read. With a pipeline, memory stays constant because only a few chunks are in flight, writing starts immediately so I see progress and failures early, and backpressure means if MongoDB slows down then reading slows down too instead of buffering up. `pipeline()` also destroys every stream in the chain and gives me a single error callback, so I do not leak connections on a failed migration."

4. Error Handling

The topic where senior answers look most different from junior ones.

4a. try/catch does not catch async errors

```
try {
  setTimeout(() => {
    throw new Error("boom");
  }, 100);
} catch (err) {
  console.log("caught it");     // never runs — process crashes
}
```

> **THE SHOP**
> The `try` block is Ramesh standing there with his hands out, ready to catch anything that falls. `setTimeout` puts the job on a stove and **Ramesh walks away**. 100ms later the stove throws something. **Nobody is standing there.** It hits the floor.

The same thing in stack terms

The call stack is a pile of plates. A function starts, a plate goes on. It returns, the plate comes off. **A try/catch lives on a plate. It can only catch things thrown while that plate is still on the pile.**

```
Time 0ms                    Time 1ms             Time 100ms
[ setTimeout  ]             (empty)              [ callback ]  <- throws
[ try/catch   ]  <- here                         (empty below)
[ main        ]
```

By the time the callback runs, the plate holding the `catch` is **gone**. The error looks down the stack for a handler and finds nothing. Crash.

The fix — catch inside the callback

```
setTimeout(() => {
  try {
    throw new Error("boom");
  } catch (err) {
    console.log("caught it");   // works
  }
}, 100);
```

Why await is different

```
try {
  await db.orders.insertOne(order);   // caught correctly
} catch (err) { }
```

`await` **does not let the plate come off**. The function is paused but its frame is preserved — Node stores it and puts it back on the stack when the promise resolves. So when the error arrives, the try/catch is still below it.

> **Remove the `await` and it breaks again.**
>
> ```
> try {
>   db.orders.insertOne(order);        // no await — escapes
> } catch (err) { }
> ```
>
> A missing `await` **silently turns a caught error into an unhandled one**. Extremely common bug.

4b. Unhandled rejections

Name	Comes from
`uncaughtException`	a `throw` nobody caught
`unhandledRejection`	a rejected promise nobody caught

Same problem — an error with nobody standing there. Two different sources.

Before Node 15: printed a warning, kept running. **Node 15 and after: the process crashes.** This changed deliberately — an unhandled rejection means you have a bug, and Node decided crashing is safer than pretending.

YOUR QUESTION — WHERE DOES THIS CODE LIVE?

One place, in your entry file — `server.js`, `index.js` or `main.ts`. Registered **once** at startup, applies to the whole process. Not per-route, not per-file. It is a process-level hook.

```
// server.js
const server = app.listen(3000);

process.on("unhandledRejection", handler);
process.on("uncaughtException",  handler);
process.on("SIGTERM",            shutdown);
```

In practice you would put these in an `errorHandlers.js` and call one `registerErrorHandlers(server)` from the entry file.

4c. Crash, do not swallow

Most candidates say "log it and keep the server running." That is wrong, and interviewers listen for it.

```
// what juniors write
process.on("uncaughtException", (err) => {
  console.error(err);
  // keep going
});
```

YOUR QUESTION — WHAT IF I REMOVE PROCESS.EXIT(1)?

The process keeps running. Registering the handler overrides Node's default crash behaviour. Your log line prints and the server carries on. That looks like a fix. It is a **silence**.

Say this rejected: `db.orders.insertOne(order)` with no await. **The order was never saved** — but the request already returned `200 OK`. Priya sees "Order placed successfully." There is no order in the database.

Now that happens 400 times over the weekend. The process never restarted, no alert fired, and you find out on Monday from customer complaints.

An uncaught exception means the process is in an unknown state. A transaction may be half-done, a lock may be held, a counter may be wrong. That is what "unhandled" means. Continuing means serving requests from a process you cannot reason about.

```
// what you should write
process.on("uncaughtException", (err) => {
  logger.fatal(err);                      // log it first
  server.close(() => process.exit(1));    // drain, then die
  setTimeout(() => process.exit(1), 10000).unref();   // force exit if hung
});
```

Then PM2, Kubernetes or systemd restarts you in a clean state. A 2-second restart is cheap. Six hours of silently dropping orders is not.

YOUR QUESTION — WHAT DOES SERVER.CLOSE() DO AND WHEN IS IT CALLED?

It is called **immediately**, but the exit only happens once existing requests finish.

`server.close()` **stops accepting new connections** while letting in-flight requests complete. Priya's request is mid-way through saving her order. Without `close()`, `process.exit(1)` kills it instantly and her order is lost mid-write. With it: the port stops accepting, Priya's request completes, the callback fires, then the process exits.

The catch: if a request hangs, `close()` never calls back and you never exit. Hence the 10-second force-exit timer alongside it.

4d. Operational vs programmer errors

	Operational	Programmer
What	Expected failures	Bugs
Examples	MongoDB timed out, invalid user input, 503 from a payment gateway, file not found	`undefined is not a function`, missing `await`, reading a property of `null`
Response	**Handle it** — retry, return 400, circuit break	**Crash** — you cannot recover from a bug

THE SHOP

Operational: the milk delivery was late. The shop is fine. Ramesh says "sorry, no chai today" and keeps serving.

Programmer: Ramesh has forgotten how to make chai. Do not let him keep serving customers. Send him home and get a fresh Ramesh.

The test: ask "did I expect this could happen?" Yes means operational, handle it. No means programmer error, let it crash.

SAY THIS

"Operational errors I handle explicitly — retries, timeouts, proper status codes. Programmer errors I let crash, because a bug means my assumptions are wrong and I cannot safely reason about the process state."

4e. Express does not catch async errors

```
app.get("/orders", async (req, res) => {
  const orders = await db.orders.find().toArray();   // throws
  res.json(orders);
});

app.use((err, req, res, next) => {
  res.status(500).json({ error: "failed" });         // never runs in Express 4
});
```

You wrote an error handler. It does not fire. The request just **hangs** until timeout.

Why: Express 4 predates async/await. It wraps your handler in a try/catch, which only catches **synchronous** throws. An async function does not throw — it **returns a rejected promise**. Express 4 ignores the return value, so the rejection floats away.

Fix 1 — wrap it

```
const asyncHandler = (fn) => (req, res, next) =>
  Promise.resolve(fn(req, res, next)).catch(next);

app.get("/orders", asyncHandler(async (req, res) => {
  const orders = await db.orders.find().toArray();
  res.json(orders);
}));
```

Or use the `express-async-errors` package — one import, patches everything.

Fix 2 — Express 5

YOUR QUESTION — SHOW ME WHAT EXPRESS 5 DOES INTERNALLY

Express 4's router, simplified:

```
try {
  handler(req, res, next);        // return value ignored
} catch (err) {
  next(err);                      // only catches sync throws
}
```

Express 5's router:

```
const result = handler(req, res, next);

if (result && typeof result.then === "function") {
  result.catch(next);             // the new line
}
```

That is the whole fix. Express 5 checks whether your handler returned a promise, and if so attaches `.catch(next)` itself — exactly what your wrapper was doing manually.

And if it is not a promise? The `if` is skipped and nothing changes. A normal non-async handler returns `undefined`, so Express falls back on its outer try/catch, which already handles synchronous throws. **Both cases are covered** — Express 5 simply added the missing half.

NestJS never had this problem. Its exception filters handle async errors out of the box. Worth mentioning, since WeKan list NestJS in the job description.

4f. The EventEmitter error event

```
emitter.emit("data",  chunk);   // nobody listening — nothing happens
emitter.emit("close");          // nobody listening — nothing happens
emitter.emit("error", err);     // nobody listening — THROWS
```

`error` is the **only** event with this behaviour. Node deliberately made it loud so a failing stream cannot fail silently.

YOUR QUESTION — DOES A MISSING FILE KEEP READING CHUNKS UNNECESSARILY?

No — the opposite. **No data is read at all.**

The file does not exist, so there is nothing to read. Node tries to open it, the OS returns `ENOENT`, the stream emits `error`, no listener exists, it throws, the process crashes. **The `data` event never fires even once.**

`error` does not happen *alongside* reading — it **replaces** it. File exists: data, data, data, end. File missing: error, and that is the end of it.

So the missing listener does not cost wasted reading. It costs your entire process. One wrong file path drops every user on that server. With the listener, one request gets a 404 and the server keeps running — the listener turns a process-wide crash into a handled operational error.

```
fs.createReadStream("missing.csv")
  .on("data",  handleChunk)
  .on("error", (err) => res.status(404).json({ error: "File not found" }));
```

Everything that is an EventEmitter needs this: file streams, HTTP responses, sockets, MongoDB connections, RabbitMQ channels.

4g. Graceful shutdown

Kubernetes deploys a new version and sends **SIGTERM** to your old pod. **Without a handler, Node dies instantly** — every in-flight request drops. Deploy ten times a day and that is ten batches of angry customers.

```
process.on("SIGTERM", async () => {
  server.close();                   // stop accepting new requests
  await drainInFlight();            // let current ones finish
  await mongoClient.close();        // close the DB pool
  process.exit(0);
});

setTimeout(() => process.exit(1), 25000).unref();   // hard limit
```

Kubernetes gives a grace period, **30 seconds by default**, then sends SIGKILL which cannot be caught. Your shutdown must finish inside that window.

Signal	Sent by	Catchable
SIGTERM	Kubernetes, PM2, `docker stop`	**Yes**
SIGINT	Ctrl+C	**Yes**
SIGKILL	`kill -9`, Kubernetes timeout	**Never**

5. Memory and Garbage Collection

The question: "*your Node service is using 3 GB of RAM, how do you debug it?*"

5a. How the heap is organised

V8 splits the heap into two areas:

- **New Space (young generation)** — small, 1–8 MB. Every new object is born here.
- **Old Space (old generation)** — large, hundreds of MB to GB. Survivors get promoted here.

Why two? Because most objects die young. An order object exists for the few milliseconds of a request, then nothing points to it. That is true of 99% of objects your app creates.

So New Space is cleaned **very frequently and very fast** — almost everything there is already garbage, so there is little to copy. If an object **survives two cleanups**, V8 concludes it is sticking around and promotes it to Old Space.

	Scavenge (minor GC)	Mark-Sweep (major GC)
Cleans	New Space	Old Space
Frequency	Every few seconds	Rarely
Duration	~1ms	50ms to over 1s
Blocks the loop?	Barely	**Yes**

> **SAY THIS**
> "V8 uses generational garbage collection — objects start in New Space, which is collected frequently and cheaply, and survivors are promoted to Old Space. Minor GC is negligible; major GC stops the event loop, so a growing Old Space shows up as **latency spikes before it shows up as a crash**."

5b. The four causes of leaks

A leak is not "forgot to free memory" — JavaScript has no free. A leak is **something still points to an object you are done with**, so GC cannot collect it.

1. The growing global cache

```
const cache = {};

app.get("/order/:id", async (req, res) => {
  if (!cache[req.params.id]) {
    cache[req.params.id] = await db.orders.findOne({ _id: req.params.id });
  }
  res.json(cache[req.params.id]);
});
```

Looks smart. But `cache` **never removes anything**. Every order ever requested stays in memory forever.

> **YOUR QUESTION — EXPLAIN THE LRU-CACHE FIX**
> **LRU = Least Recently Used.** A cache that **throws things out** when full, evicting whatever you touched longest ago.

```
const { LRUCache } = require("lru-cache");

const cache = new LRUCache({
  max: 1000,              // never hold more than 1000 orders
  ttl: 1000 * 60 * 5      // also expire anything after 5 minutes
});

let order = cache.get(id);
if (!order) {
  order = await db.orders.findOne({ _id: id });
  cache.set(id, order);
}
```

Walk it with `max: 3`:

```
set A   ->  [A]
set B   ->  [A, B]
set C   ->  [A, B, C]     full
get A   ->  [B, C, A]     A moves to newest
set D   ->  [C, A, D]     B evicted — least recently used
```

Memory is now **bounded**. 1000 orders maximum, forever, no matter how much traffic. The `ttl` adds a second guard so even a hot key expires and you do not serve stale data.

Redis alternative: same idea, but the cache lives *outside* your process — shared across all pods and it never touches Node's heap. Better for multi-instance deployments.

> **REMEMBER THIS PHRASING**
> A cache without an eviction policy is a memory leak with extra steps.

2. Listeners you never remove

YOUR QUESTION — SHOW REMOVELISTENER IN CODE

The leak:

```
app.get("/orders", (req, res) => {
  mongoClient.on("close", () => cleanup(res));   // added every request
  // never removed
});
```

10,000 requests means 10,000 listeners, each closure holding a `res` object.

Fix A — remove it when done:

```
app.get("/orders", async (req, res) => {
  const onClose = () => cleanup(res);
  mongoClient.on("close", onClose);

  try {
    const orders = await db.orders.find().toArray();
    res.json(orders);
  } finally {
    mongoClient.removeListener("close", onClose);   // always removed
  }
});
```

You must pass the same function reference. This does nothing:

```
emitter.on("close", () => cleanup());
emitter.removeListener("close", () => cleanup());   // different function!
```

Two identical-looking arrow functions are two different objects. That is why you name it `onClose` and reuse the variable.

Fix B: `mongoClient.once("close", onClose)` — auto-removes after firing.
Fix C, the real one: do not register per-request listeners at all. Register once at startup.

Node's built-in leak detector:

```
MaxListenersExceededWarning: 11 close listeners added
```

The warning at 11 is **not a limit** — it is Node guessing you are adding in a loop. You can raise it with `setMaxListeners(20)`, but check first, because usually it is this bug.

3. Closures holding more than you think

```
function setup() {
  const bigData = new Array(1e6).fill("x");   // ~8 MB

  return function log() {
    console.log("done");                      // does not use bigData
  };
}
const fn = setup();   // bigData may stay alive
```

The returned function keeps its parent scope alive. V8 often optimises this away, but with a `setInterval` or an event listener holding the closure, that 8 MB never goes.

4. Timers never cleared

YOUR QUESTION — DO TIMERS STAY IN THE HEAP AFTER EXECUTION?

`setTimeout` **— no.** It fires once, then it is gone and everything it held becomes collectable.

```
setTimeout(() => refresh(bigObject), 1000);
// after 1 second: fires, released, bigObject collectable
```

`setInterval` **— yes, forever.** It never "finishes." It is scheduled to fire again, so it stays alive, and so does everything its callback references.

```
setInterval(() => refresh(bigObject), 1000);
// bigObject alive forever
```

The fix:

```
const timer = setInterval(() => refresh(bigObject), 1000);
process.on("SIGTERM", () => clearInterval(timer));
```

The one setTimeout case that does leak — when it reschedules itself:

```
function poll() {
  checkStatus(bigObject);
  setTimeout(poll, 1000);      // never ends — same as setInterval
}
```

THE RULE

A pending timer is a **GC root**. Anything reachable from its callback cannot be collected while the timer is scheduled.

All four are the same bug: an unbounded collection that only grows. Cache with no eviction. Listener array with no removal. Closure scope never released. Timer never cleared.

The question to ask out loud: "What in this code grows with traffic and never shrinks?"

5c. Debugging a leak in production

Step 1 — confirm it is actually a leak

```
setInterval(() => {
  const m = process.memoryUsage();
  logger.info({
    heapUsed: (m.heapUsed / 1024 / 1024).toFixed(0) + " MB",
    rss:      (m.rss      / 1024 / 1024).toFixed(0) + " MB"
  });
}, 60000);
```

- `heapUsed` — your JavaScript objects
- `rss` (Resident Set Size) — total process memory, including Buffers and native memory

Read the shape over hours. Sawtooth — up, drop, up, drop — is **healthy**, GC is working. A staircase that only climbs is a **leak**.

And check which number is growing. If `heapUsed` is flat but `rss` climbs, the leak is in **Buffers**, not JavaScript objects — different problem, and a heap snapshot will not find it.

Step 2 — heap snapshots

YOUR QUESTION — WHERE IS THE SNAPSHOT TAKEN?

From your Node server, not from a browser. Chrome DevTools is only the **viewer** — a file reader. Nothing to do with browsers or users.

```
const v8 = require("v8");
v8.writeHeapSnapshot("/tmp/snap-1.heapsnapshot");
```

This runs **inside your Node process on the server** and dumps that process's entire heap to a file on that server's disk. Then `scp` or `kubectl cp` it down, open Chrome, DevTools, Memory tab, Load. **You are not connected to anything** — DevTools is just parsing a file you downloaded.

And it is not per-user. It is the whole process heap — every object from every request, plus caches, connection pools and module-level state. That is exactly what you want, since a leak accumulates *across* requests.

Triggering it in production:

```
process.on("SIGUSR2", () => {
  v8.writeHeapSnapshot(`/tmp/snap-${Date.now()}.heapsnapshot`);
});
// then from the box:  kill -SIGUSR2 <pid>
```

Two warnings worth saying aloud: writing a snapshot **pauses the process** — a 2 GB heap can freeze for several seconds, so do it on one pod, not all. And **the file is roughly heap-sized**, so check disk space.

Take two snapshots, an hour apart, under load. Open both in DevTools and use the **Comparison** view.

This is the key move. One snapshot shows what is in memory, which is overwhelming. **Two snapshots show what is *growing*** — and that is your leak. Sort by Delta, look at the top row, then click into **Retainers** to see *what is still holding it*.

Step 3 — match the retainer to the cause

Retained by	Cause and fix
A plain object	Unbounded cache — add `lru-cache`
`_events`	Listener leak — `removeListener` or `once`
A Timeout	Uncleared interval — `clearInterval`

The escape hatch that is not a fix.

```
node --max-old-space-size=4096 server.js
```

This raises the heap limit to 4 GB. It **buys time, it fixes nothing** — you will hit 4 GB next week instead of today. Say explicitly: "I would raise the limit only to stop the bleeding while I take snapshots, not as a fix." That framing matters.

6. EventEmitter

The idea: something happens, announce it. Whoever cares, listens.

```
class OrderService extends EventEmitter {
  placeOrder(order) {
    db.orders.insertOne(order);
    this.emit("orderPlaced", order);      // announce it
  }
}

orders.on("orderPlaced", (o) => sendEmail(o));
orders.on("orderPlaced", (o) => updateInventory(o));
orders.on("orderPlaced", (o) => notifyWarehouse(o));
```

`placeOrder` does not know or care who is listening. Add a fourth listener tomorrow and `placeOrder` never changes. **That decoupling is the whole point.**

Why it matters

Almost everything in Node is an EventEmitter — streams, HTTP servers, sockets, the process object. So the pattern is not a nice-to-have, it is the foundation Node is built on.

The thing people get wrong: emit is synchronous

```
console.log("A");
orders.emit("orderPlaced", order);   // all 3 listeners run right here
console.log("B");
```

Output: A, then all three listeners, then B. It is **not queued**. It is effectively a for-loop calling each listener in registration order, on the current stack.

> **Consequence 1:** a slow listener blocks everything. You emitted expecting fire-and-forget and you froze the shop. If a listener is heavy, push it to a queue instead.
>
> **Consequence 2:** a listener that throws crashes the emitter's caller, since it is on the same stack.

> **SAY THIS**
> "EventEmitter is the pub-sub foundation Node is built on — streams, sockets and the HTTP server are all EventEmitters. The key detail is that `emit` is **synchronous**: listeners run on the current stack in registration order, so a slow listener blocks the event loop and a throwing listener crashes the emitter's caller."

7. Cluster

The problem: your box has 8 CPU cores. Your Node process uses **one**. Ramesh is one waiter and the shop has 8 counters, 7 of them empty. You are paying for 8 cores and using 12.5%.

```
const cluster = require("cluster");
const os = require("os");

if (cluster.isPrimary) {
  for (let i = 0; i < os.cpus().length; i++) cluster.fork();   // 8 processes
  cluster.on("exit", () => cluster.fork());                    // restart on death
} else {
  app.listen(3000);                                            // each worker listens
}
```

"How can 8 processes share port 3000?"

They do not. **Only the primary holds the port.** It accepts each incoming connection and hands it to a worker, round-robin by default on Linux. Workers never bind the port — they receive already-accepted sockets.

The three things that make this a senior answer

1. Cluster does not fix blocking. Your p99 is 2 seconds because of a heavy loop. Add 8 workers: p99 is **still 2 seconds**. You now handle 8 blocked requests concurrently instead of 1. Cluster multiplies **throughput**, not **latency**.

2. Nothing is shared between workers. Separate processes, separate memory.

```
let counter = 0;   // each worker has its own
const cache = {};  // 8 separate caches
```

In-memory sessions break — a user hits worker 3, then worker 7, and is logged out. **Shared state must go to Redis.** This is the bug people actually hit.

3. You usually should not write this code. PM2 does it with `pm2 start server.js -i max`, plus restarts and zero-downtime reloads. In Kubernetes, run **one process per pod** and scale pods instead — clustering inside a pod fights the orchestrator over CPU limits and restarts.

YOUR QUESTIONS — SINGLE CORE, AND WHO MANAGES WORKER MEMORY

"Can a big Node app run on one core?" Yes, and it is common. A single process handles thousands of concurrent connections, because that is what the event loop is for. "Big application" means many features and lots of code — that costs **memory**, not CPU. The only thing that breaks it is CPU-bound work.

The real reason people avoid a single process is not performance, it is **resilience**: one process crashes and your service is down. You want at least two of something — cluster workers or Kubernetes pods.

"Do 4 workers need an orchestrator to manage memory?" No — **nobody manages their memory**. Each worker is a full independent Node process with its own V8 heap, own GC, own limit. The primary only distributes **connections**.

The consequence: **memory multiplies**. 1 process at 200 MB becomes 4 workers at 800 MB. Code, dependencies and connection pools all load 4 times. And if you have a leak, all four leak independently.

PM2 and Kubernetes manage the **process lifecycle**, not memory — restarting crashed workers, zero-downtime reloads, health checks. The closest thing to memory management is a safety net:

```
pm2 start server.js -i 4 --max-memory-restart 500M
```

WebSockets caveat: round-robin breaks WebSockets, since a connection must stay on one worker. You need **sticky sessions** (route by client IP) or a Redis adapter to pass messages between workers.

8. Buffers

A Buffer is raw bytes. Not text, not objects — just bytes.

```
const buf = Buffer.from("Rinish");
console.log(buf);          // <Buffer 52 69 6e 69 73 68>
console.log(buf.length);   // 6
```

JavaScript strings are text. But a file, an image, a TCP packet, an encrypted payload — those are **binary**. Buffer is how Node holds them. Every chunk in that 2 GB CSV export was a Buffer.

Buffers live outside the V8 heap

V8's heap is for JavaScript objects and has a default limit of roughly 2 GB on 64-bit. **Buffers are allocated in native memory, outside that heap.**

YOUR QUESTION — "OUTSIDE" WHERE, EXACTLY?

Still inside your process. Same process, same RAM. Node's memory has several regions and V8's heap is only one of them.

The region is called the **native heap**, also the C++ heap or malloc heap. Ordinary operating-system memory, requested by Node's C++ layer via `malloc()`.

V8's heap is special: V8 reserves a big region upfront and manages it itself, with GC moving objects around inside it. The native heap is ordinary — allocate a block, free a block.

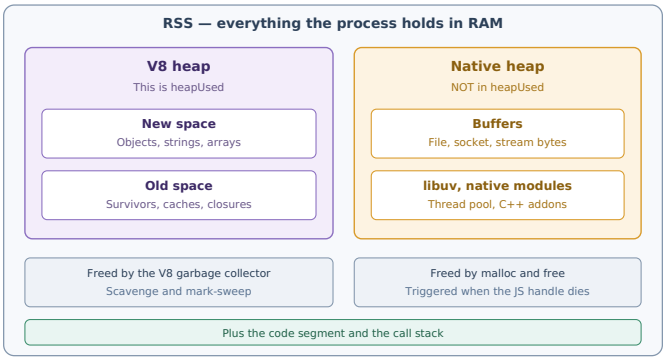

Figure 5 — Process memory regions. This is why RSS can grow while heapUsed stays flat.

How a Buffer actually works

`const buf = Buffer.from(data)` creates **two** things:

1. A **small JS object** in the V8 heap — the handle you hold. Maybe 100 bytes.
2. The **actual bytes** in the native heap. Could be 50 MB.

Freeing: when the handle becomes unreachable, V8's GC collects it, and a callback then frees the native block. So Buffers *are* cleaned up — but only as a side effect of the handle dying.

Which is why Buffer leaks are sneaky. Holding a 100-byte handle in an array keeps 50 MB alive. Your heap snapshot shows 100 bytes. RSS shows 50 MB.

```
const chunks = [];
stream.on("data", (c) => chunks.push(c));   // builds the whole file in RSS
```

allocUnsafe — the security gotcha

```
Buffer.alloc(1000);         // zero-filled, safe, slower
Buffer.allocUnsafe(1000);   // NOT zero-filled, fast
```

`allocUnsafe` hands you recycled memory **without wiping it**. It may contain fragments of whatever was there before — old request bodies, passwords, tokens. If you do not overwrite every byte and then send it out, **you leak old data to a user**. Use `Buffer.alloc()` unless you are immediately filling the whole thing.

Small detail: Node keeps an internal 8 KB buffer pool (`Buffer.poolSize`). Buffers smaller than 4 KB are carved from it rather than getting their own `malloc` call, because allocating is expensive.

SAY THIS

"Buffers hold raw binary and are allocated in the native heap, outside V8's managed heap. The JS side holds only a small handle — so RSS reflects the real size while `heapUsed` and heap snapshots only see the handle. **That is the first thing I check on a memory issue:** if RSS grows while heap is flat, it is Buffers, and a heap snapshot will not find it."

9. Modules — CommonJS vs ECMAScript Modules

The names, expanded.
ECMAScript is the formal name of the JavaScript language specification, maintained by Ecma International. "JavaScript" is the common name; ECMAScript is the standard. ES6, ES2020 and so on are its versions.
ESM = ECMAScript Modules — the official module standard, `import` / `export`.
CJS = CommonJS — Node's original module system, `require` / `module.exports`. Predates ESM.

```
// CommonJS
const express = require("express");
module.exports = { placeOrder };

// ECMAScript Modules
import express from "express";
export { placeOrder };
```

Which system a file uses: `.cjs` is always CJS, `.mjs` is always ESM, and `.js` depends on whether `"type": "module"` is set in `package.json`.

The one difference that matters

`require` is synchronous. `import` is asynchronous.

```
if (isProduction) {
  const config = require("./prod-config");   // works, runs at this line
}

if (isProduction) {
  import config from "./prod-config";        // syntax error
}
```

ESM imports are **hoisted and resolved before any code runs**. They must be top-level — you cannot conditionally import.

The require cache — practically useful

```
const a = require("./db");
const b = require("./db");
a === b;   // true — same object
```

Node caches modules by resolved path. The file runs **once**; every later `require` gets the same exports object. This is why module-level code is effectively a singleton:

```
// db.js
const client = new MongoClient(url);   // runs exactly once
module.exports = client;
```

Every file that requires it shares one connection pool. Without knowing about the cache you would wrongly assume you are creating a pool per import. ESM behaves the same way — modules evaluate once.

Circular dependencies

`a.js` requires `b.js`, which requires `a.js`. CJS does not crash — it returns a **partially filled** exports object, so you get `undefined` for things not yet assigned. Confusing bugs, no error message.

The real fix is design, not a trick: extract the shared piece into a third module.

YOUR QUESTION — CAN I USE BOTH AT ONCE?

Yes, but **the two directions are not equal**.

ESM importing CJS — works fine. Because ESM loading is async, it can load a CJS module and wait. Almost every npm package is still CJS, so this direction had to work.

```
import { Router } from "express";     // may not work
import express from "express";
const { Router } = express;           // do this instead
```

Named imports often fail because CJS exports are decided at runtime, so Node cannot always see them statically.

CJS importing ESM — this is the broken direction.

```
const chalk = require("chalk");   // ERR_REQUIRE_ESM if chalk is ESM-only
```

`require` is synchronous and ESM loading is asynchronous. A sync function cannot wait for an async operation. That is the fundamental blocker.

The workaround — dynamic import:

```
async function run() {
  const chalk = await import("chalk");   // works from CJS
  console.log(chalk.default.green("ok"));
}
```

Note `.default` — that is where the default export lands. *(Node 22+ has begun allowing `require()` of some ESM modules, but do not rely on it.)*

In one project: set a default with `"type"` in package.json and override per file with `.mjs` or `.cjs`.

SAY THIS

"You can mix them. ESM importing CJS works because ESM loading is async, though named imports may need the default-then-destructure workaround. The other direction is the problem: `require()` is synchronous and cannot load an async ESM module, so you need `await import()`. **In practice I would keep a codebase on one system and only mix when a dependency forces it**, since mixed codebases cause confusing tooling bugs in Jest, TypeScript and bundlers."

10. Timers

10a. setTimeout(fn, 100) is a minimum, not exact

> **THE SHOP**
> You order chai and Ramesh says "3 minutes." But when the 3 minutes are up, Ramesh is busy with another customer. So he brings your chai at 5 minutes. **The stove was ready at 3 minutes. Ramesh was not.**

```
setTimeout(() => console.log("fired"), 100);

for (let i = 0; i < 1e9; i++) {}     // takes 500ms

console.log("loop done");

// loop done   <- at ~500ms
// fired       <- at ~500ms, not 100ms
```

The timer *was* ready at 100ms and its callback went into the queue. But Ramesh was counting coins and did not reach the timers phase until 500ms.

Meaning: `setTimeout(fn, 100)` means "not before 100ms." Could be 100, could be 500. It depends on how busy the thread is — which is another reason event loop lag matters.

10b. setInterval drifts

You want a report every 1 second. `sendReport` takes 300ms.

What you expect	What you get
0s, 1s, 2s, 3s	0.0s, 1.3s, 2.6s, 3.9s

The gap is **1.3 seconds**, not 1, and it keeps sliding. After an hour you have drifted by minutes.

Why: `setInterval` waits 1 second *after your callback finishes*, not from when it started. The 300ms is added on top, every time.

```
function tick(expected = Date.now() + 1000) {
  sendReport();
  const drift = Date.now() - expected;
  setTimeout(() => tick(expected + 1000), Math.max(0, 1000 - drift));
}
tick();
```

> **The bigger setInterval trap.** If the callback is async and slower than the interval, **runs overlap** — ten pending database calls stack up. Self-scheduling with `setTimeout` *after* the work completes avoids this entirely, which is why cron-style jobs should use setTimeout chains, not setInterval.

10c. unref()

Node stays alive as long as it has something pending.

```
setInterval(() => console.log("check"), 5000);
// this script NEVER exits
```

```
const timer = setInterval(() => console.log("check"), 5000);
timer.unref();
// this script exits immediately
```

`unref()` means: **"run this timer if the process is alive, but do not stay alive just for it."**

Where you actually want it — the shutdown guard:

```
server.close(() => process.exit(0));                // normal exit
setTimeout(() => process.exit(1), 10000).unref();   // force exit if stuck
```

Without `unref()`, that safety net would itself keep the process alive for 10 seconds even after a clean shutdown — the exact opposite of what you want.

11. MongoDB — Indexes and Query Performance

WeKan Enterprise Solutions is a **strategic partner of MongoDB**, doing complex data migrations and legacy modernisation, working directly with MongoDB Solutions Architects. **MongoDB depth is the differentiator in this interview**, not Node.js alone.

11a. What an index actually is

YOUR QUESTION — WHAT DID "REGISTER" MEAN?

That was my word, not a MongoDB term. I meant a **register book** — a shop ledger, the thick notebook where a shopkeeper writes entries in order.

That is all an index is: **a notebook kept in sorted order, separate from the actual data**.

Your `orders` collection is the pile of order slips, in whatever order they arrived. The index on `amount` is a notebook where every amount is already written in sorted order, each with a pointer to the real slip:

```
Rs 48,900  ->  slip #9921
Rs 47,200  ->  slip #1043
Rs 46,850  ->  slip #7788
```

MongoDB maintains this notebook automatically on every insert and update. "Read the register" means "walk the index."

11b. A sort without a supporting index

```
db.orders.find({}).sort({ amount: -1 }).limit(10)
```

Without an index, MongoDB must read all 5 lakh documents, load them into memory, sort all 5 lakh, throw away 4,99,990, and return 10. This is a **blocking in-memory sort** — blocking because it cannot return the first result until it has seen the last document.

And it can simply fail. MongoDB caps in-memory sorts at **32 MB**:

```
Sort exceeded memory limit of 33554432 bytes
```

Your endpoint returns a 500. This is a real production error, not a theoretical one.

With an index, the same query means: open the notebook, read the first 10 lines, stop. No scanning, no sorting, no 32 MB risk.

SAY THIS

"A sort without a supporting index becomes a blocking in-memory sort, capped at 32 MB. With the index it is an `IXSCAN` — MongoDB walks the pre-sorted index and stops at the limit."

11c. Index direction

YOUR QUESTION — WHAT IF THE SORT IS ASCENDING?

Same index still works. **A single-field index can be read in both directions** — to get the cheapest orders MongoDB just reads the notebook from the bottom upward. So `createIndex({ amount: -1 })` supports both `sort({ amount: -1 })` and `sort({ amount: 1 })`. You do not need two indexes.

Where direction does matter: compound indexes. Given `{ city: 1, amount: -1 }`:

```
.sort({ city: 1,  amount: -1 })   works — read straight down
.sort({ city: -1, amount:  1 })   works — read straight up, everything flips
.sort({ city: 1,  amount:  1 })   FAILS — in-memory sort
```

The rule: you can read an index forwards or backwards, but you must **flip every field, or none**. Mixed directions need their own index.

11d. The ESR rule

The single most-asked MongoDB index question.

```
db.orders.find({ city: "Jamshedpur", amount: { $gt: 5000 } })
         .sort({ createdAt: -1 })
```

Three fields. Which order in the index? **E, then S, then R:**

```
db.orders.createIndex({ city: 1, createdAt: -1, amount: 1 })
```

Letter	Means	Why it goes there
E	Equality	All Jamshedpur rows sit in one continuous block. MongoDB jumps straight to it and ignores the rest of the notebook. Huge cut.
S	Sort	Inside that block, rows are already newest-first. MongoDB reads them in order — **no sort stage at all**. This is the key win.
R	Range	Now it simply skips rows where amount is too low as it walks.

What breaks if range goes in the middle (E, R, S): a range match gives **many** amount values. Within each one dates are sorted, but *across* them the dates are jumbled. So MongoDB must collect all matches and sort them in memory. You are back to a blocking sort.

> **THE REASON, IN ONE LINE**
> A range spreads your results across many index values, so any sort field placed after it is no longer in order.

11e. explain()

> **YOUR QUESTION — WHAT DO I ACTUALLY GET FROM EXPLAIN()?**
> Add `.explain("executionStats")` to any query. That is the mode you want — the default only shows the plan, not what happened. The output is a large JSON blob; **ignore 95% of it**. Four fields matter.

```
db.orders.find({ city: "Jamshedpur" })
         .sort({ amount: -1 })
         .explain("executionStats")
```

Field	Tells you	Good value
`stage`	How it found the data	`IXSCAN`
`totalDocsExamined`	Documents read	close to nReturned
`nReturned`	Documents returned	—
`executionTimeMillis`	Time taken	low

```
BAD                                GOOD
"nReturned": 10                    "nReturned": 10
"totalDocsExamined": 500000        "totalDocsExamined": 10
"executionTimeMillis": 812         "executionTimeMillis": 2
"stage": "SORT"                    "stage": "FETCH"
  "inputStage": "COLLSCAN"           "inputStage": "IXSCAN"
```

Stage	Meaning
`COLLSCAN`	**Read every document. Bad on large collections.**
`IXSCAN`	**Walked the index. Good.**
`SORT`	**Sorted in memory. Your index does not support the sort.**
`FETCH`	Went from index entry to the real document. Normal.

Bonus: `IXSCAN` with **no** `FETCH` is a **covered query** — the index alone had every field needed, so MongoDB never touched the documents. Fastest possible. Worth naming in an interview.

> **SAY THIS**
> "I would run `explain('executionStats')` and check that `totalDocsExamined` is close to `nReturned`, and that there is no `SORT` stage. A large gap between examined and returned means the index is not being used properly."

11f. Other index rules

Leftmost prefix. An index on `{ city, amount, createdAt }` also works as an index on `{ city }` and `{ city, amount }` — but **not** on `{ amount }` alone. Entries are grouped by city first; without the city you cannot jump to a block.

Practical value: one well-ordered compound index often replaces three separate ones. This is the answer to "you have 8 indexes on this collection, how would you reduce them?"

Multikey indexes (arrays). An index on an array field creates **one entry per array element**. You cannot have a compound index on two array fields, and multikey indexes cannot fully cover a query.

Indexes cost writes. Every index must be updated on every insert and update. Eight indexes means eight extra writes per insert.

11g. $indexStats — finding dead indexes

```
db.orders.aggregate([{ $indexStats: {} }])
```

Returns one entry per index with an `ops` count — how many queries used it — and a `since` timestamp.

```
{ name: "customerEmail_1",
  accesses: { ops: 0, since: "2026-06-01T00:00:00Z" } }   // never used
```

Zero ops in two and a half months. Somebody added it for a feature that was removed. It is still slowing every insert and consuming RAM. Drop it.

TWO TRAPS — THE SECOND IS THE GOOD ONE

1. Counters reset on restart. `ops` counts since the last `mongod` start. If the server restarted yesterday, everything looks unused. **Always check `since`** before concluding an index is dead.

2. Zero does not always mean useless. A monthly reporting query or a quarterly reconciliation job may legitimately show 0 ops today. Check with the team before dropping.

11h. Are indexes in RAM or on disk?

YOUR QUESTION — ANSWERED

Both — and that is the whole point.

Indexes live **on disk** permanently; that is the source of truth and survives restarts. But MongoDB cannot search a disk file directly — to use an index it must **load those pages into RAM**.

MongoDB reserves a chunk of RAM called the **WiredTiger cache**, by default **50% of (RAM minus 1 GB)**. On a 16 GB machine, roughly 7.5 GB. **Documents and indexes compete for space in that same cache.**

Warm — index page already in cache — served from RAM, microseconds. **Cold** — read from disk, evict something else, then use it — milliseconds. **100 to 1000 times slower.**

Where the dead index hurts. Cache is 7.5 GB, working indexes need 6 GB, everything stays warm. Add a 2 GB unused index and total is 8 GB. MongoDB now **evicts constantly** — a live index page is thrown out, a query needs it back, something else is thrown out. This is **cache thrashing**, and p99 latency jumps.

So the dead index is not just idle overhead. **It is actively pushing your working indexes out of RAM.**

```
db.orders.totalIndexSize()      // bytes, all indexes on this collection
db.orders.stats().indexSizes    // per-index breakdown
```

SAY THIS — THE PHRASE THEY LISTEN FOR

"Indexes are stored on disk but must be paged into the WiredTiger cache to be used. The rule of thumb is that **your working set — hot indexes plus hot documents — should fit in that cache**. Once indexes spill to disk you get cache eviction and latency falls off a cliff, which is why unused indexes are a real cost and not just clutter."

Appendix A — Acronyms expanded

Short	Full form	Meaning
ECMAScript	—	The formal name of the JavaScript language specification, maintained by Ecma International. ES6, ES2020 etc. are its versions.
ESM	ECMAScript Modules	The official JavaScript module standard — `import` / `export`
CJS	CommonJS	Node's original module system — `require` / `module.exports`
V8	—	Google's JavaScript engine, named after the car engine
libuv	library for Unified asynchronous I/O	The C library that gives Node its event loop and thread pool
I/O	Input / Output	Reading and writing — files, network, disk
epoll	event poll	Linux kernel API for watching many sockets at once
kqueue	kernel queue	The macOS and BSD equivalent of epoll
IOCP	I/O Completion Ports	The Windows equivalent of epoll
GC	Garbage Collection	Automatic freeing of unreachable objects
RSS	Resident Set Size	Total process memory held in RAM — heap plus native plus code
LRU	Least Recently Used	Cache eviction policy — discard what was touched longest ago
TTL	Time To Live	How long an entry stays valid before expiring
ENOENT	Error: NO ENTry	File or directory not found
SIGTERM	Signal: Terminate	Polite shutdown request. Catchable.
SIGINT	Signal: Interrupt	Ctrl+C. Catchable.
SIGKILL	Signal: Kill	Immediate termination. Cannot be caught.
SIGUSR2	Signal: User-defined 2	Free for your own use — commonly used to trigger heap dumps
PM2	Process Manager 2	Node process manager — clustering, restarts, reloads
ESR	Equality, Sort, Range	MongoDB compound index field-ordering rule
IXSCAN	Index Scan	explain() stage — MongoDB used an index. Good.
COLLSCAN	Collection Scan	explain() stage — MongoDB read every document. Bad.
ReDoS	Regular expression Denial of Service	A regex that backtracks catastrophically and freezes the loop
CDC	Change Data Capture	Streaming database changes to keep systems in sync

Appendix B — Every one-liner in one place

Read these the morning of the interview.

Topic	The sentence
Architecture	Node is not single-threaded — the JavaScript execution is single-threaded. libuv underneath is multi-threaded, and network I/O does not even use those threads because the kernel handles it.
Event loop	The event loop continuously checks the queues of completed async work and pushes their callbacks onto the call stack, one at a time, in a fixed order of phases. Between every callback it drains the nextTick queue and then the promise microtask queue.
setImmediate	Inside an I/O callback, setImmediate always runs before setTimeout, because check is the very next phase after poll while timers requires a full lap.
CPU-bound work	First I would confirm it is CPU-bound via event loop lag. Then: can the database do it? If not, a pooled worker thread. If it is genuinely long-running, it does not belong in the request path — queue it and notify asynchronously.
Cluster	Cluster multiplies throughput, not latency — it will not fix a blocked event loop. Workers share no memory, so state has to move to Redis. In Kubernetes I would scale replicas rather than cluster inside a pod.
Streams	Backpressure is the mechanism where a slow consumer signals a fast producer to slow down. write() returning false means pause, the drain event means resume. pipe() handles this automatically.
pipeline()	pipe() does not forward errors, so every stream needs its own listener or an unhandled error event crashes the process — and a failure leaks the rest of the chain. pipeline() wires up error handling across the whole chain and reports through a single callback.
try/catch	A callback runs on a fresh, empty stack — the try frame is long gone. await preserves the frame, so the catch is still below it.
Crash policy	An uncaught exception means the process is in an unknown state, so I log it, drain in-flight requests, and exit — then let the orchestrator restart me clean. The handler is a last-resort logging hook, not a recovery mechanism.
Error types	Operational errors I handle explicitly. Programmer errors I let crash, because a bug means my assumptions are wrong and I cannot safely reason about the state.
Express	Express 4 only catches synchronous throws, so an async handler's rejection escapes and the request hangs. I would wrap handlers with a .catch(next) helper, or use Express 5 which handles it natively.
EventEmitter	error is the only EventEmitter event that throws when unhandled — every other event is a no-op. So every stream, socket and connection needs an error listener.
emit	emit is synchronous: listeners run on the current stack in registration order, so a slow listener blocks the event loop.
Shutdown	On SIGTERM I stop accepting new connections, drain in-flight requests, close the DB pool, then exit — with a hard timeout inside the orchestrator's grace period, since SIGKILL follows and cannot be caught.
GC	V8 uses generational GC. Minor GC is negligible; major GC stops the event loop, so a growing Old Space shows up as latency spikes before it shows up as a crash.
Leaks	Leaks are almost always something long-lived holding a reference — an unbounded cache, listeners added per request, or an uncleared timer. Look for any structure that grows with traffic and has no eviction path.
Debugging	Log heapUsed and rss over hours — a sawtooth is healthy, a rising staircase is a leak. Then two heap snapshots an hour apart, diffed in DevTools, sorted by delta, reading retainers.
Buffers	Buffers are allocated in the native heap, outside V8's heap. If RSS grows while heapUsed is flat, it is Buffers — and a heap snapshot will not find it.
Modules	require is synchronous and resolved at runtime; import is asynchronous and hoisted, so it must be top-level. Both cache by path, which is why a module-level DB client acts as a singleton.
Timers	Timer delays are a minimum, not a guarantee. setInterval drifts because the callback's duration is not subtracted, and async callbacks can overlap.
Mongo sort	A sort without a supporting index becomes a blocking in-memory sort, capped at 32 MB. With the index it is an IXSCAN.
ESR	Equality first because it narrows to one contiguous block. Sort next so the index delivers documents already ordered. Range last, because a range spans many index values and destroys ordering for anything after it.
explain()	I check that totalDocsExamined is close to nReturned and that there is no SORT stage. A large gap means the index is not being used properly.
Index cost	Indexes are on disk but must be paged into the WiredTiger cache. The working set — hot indexes plus hot documents — should fit in that cache.

Appendix C — Rapid-fire self test

Cover the answers. If you cannot answer in two sentences, reread that chapter.

Question	Where
Why does a MongoDB query use zero threads but `fs.readFile` uses one?	1d
Eight simultaneous bcrypt hashes at 100ms each — what happens to the fifth?	1d
Name the six event loop phases in order.	1f
Predict the output: A, setTimeout B, setImmediate C, Promise D, nextTick E, F.	1g
Why does `setImmediate` always beat `setTimeout(fn,0)` inside an I/O callback?	1h
Why does a for-loop freeze all users but `await sleep(5000)` does not?	2
You need the 10 most expensive of 5 lakh orders. What is wrong with sorting in Node?	2
Why `setImmediate` and not `nextTick` when chunking work?	2
What is backpressure, and what does `pipe()` do about it?	3
Why is `pipeline()` better than `pipe()`? Give two reasons.	3
Why does `try/catch` miss a `setTimeout` throw? Answer in stack terms.	4a
What happens if you remove `process.exit(1)` from the rejection handler?	4c
MongoDB times out. Operational or programmer error? What do you do?	4d
What exactly did Express 5 add to its router?	4e
Which EventEmitter event crashes the process when unhandled, and why only that one?	4f
What does `server.close()` do, and why is a force-exit timer needed alongside it?	4g
Why does V8 split the heap into New Space and Old Space?	5a
Name the four causes of leaks. What do all four have in common?	5b
Does a fired `setTimeout` keep its objects alive? Does `setInterval`?	5b
Where is a heap snapshot taken from, and why take two?	5c
Is `emit` synchronous or asynchronous? What are the two consequences?	6
Does cluster fix a 2-second p99 caused by a heavy loop?	7
Four cluster workers — what happens to memory usage?	7
Heap is flat, RSS is climbing. What is leaking?	8
Why can't CJS `require()` an ESM-only package?	9
Why does `setInterval(fn, 1000)` drift, and what is the fix?	10b
What does `unref()` do and where would you use it?	10c
Query: equality on status, range on createdAt, sort on amount. Build the index.	11d
Which two fields of `explain()` tell you the most, and what is a good value?	11e
Why is an unused index worse than merely useless?	11h

Still to prepare for this interview

- **NestJS** — explicitly in the job description at 2+ years. Modules, dependency injection, guards, interceptors, exception filters.
- **MongoDB data modeling** — embed vs reference, schema patterns (bucket, attribute, extended reference), transactions, change streams. This is WeKan's core business.
- **One production Node.js story with numbers** — they will probe whether your Node depth is real or surface-level, given your Java background.